

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: CSA16 **COURSE CODE:** Data Warehousing and Data Mining

COURSE OUTCOME COVERED

CO5: Apply association rule mining techniques to discover interesting relationships and patterns within large datasets

QUESTIONS: 05

Total Marks:50

Annexure A

Pseudocode Writing Task (Algorithm Design)

Criteria	Problem Understanding (2) 20%	Algorithm Design (3)	Use of Control Structures (2)	Pseudocode Structure (1)	Completeness (2)	Total(10)
<i>Weightage</i>		30%	20%	10%	20%	<i>100%</i>
<i>Q1</i>						
<i>Q2</i>						
<i>Q3</i>						
<i>Q4</i>						
<i>Q5</i>						

Code of Conduct:

I Heisenberg Raja G B(192324158) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Heisenberg Raja

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient

Annexure A

Q1. Dataset: Online Shopping Transactions

Order ID	Products Purchased
O1	Mobile, Earphones
O2	Mobile, Charger
O3	Earphones, Charger
O4	Mobile, Earphones, Charger
O5	Mobile, Earphones

Question:

Design pseudocode for the **Apriori algorithm** to discover frequent product combinations from the online shopping dataset using a specified minimum support threshold. Clearly illustrate the candidate itemset generation and pruning process at each iteration.

Q2. Dataset: Library Book Borrowing Records

Borrow ID	Books Borrowed
B1	Data Science, Python
B2	Python, Machine Learning
B3	Data Science, Machine Learning
B4	Data Science, Python, Machine Learning
B5	Python, Machine Learning

Question:

Develop pseudocode for the **FP-Growth algorithm** to identify frequent borrowing patterns without generating candidate itemsets. Explain the steps involved in constructing the FP-Tree using the given borrowing records. (BL4)

Q3. Dataset: Frequently Purchased Services

Service Combination	Support Count
{Internet} 4 {Streaming} 4	
{Cloud Storage} 3 {Internet, Streaming} 3 {Streaming, Cloud Storage} 2	

Question:

Write pseudocode to generate **association rules** from the given frequent service combinations. Include the procedures for confidence calculation, rule generation, and filtering based on a minimum confidence threshold. (BL3)

Q4. Dataset: Student Course Enrollment**Student ID Courses Enrolled**

S1	Python, Database
S2	Python, AI
S3	Database, AI
S4	Python, Database, AI
S5	Python

Question:

Design pseudocode for **constraint-based frequent itemset mining** where only itemsets containing the course "**Python**" are considered during the mining process. Explain how the constraint affects candidate generation and pruning. (BL4)

Q5. Dataset: Hospital Treatment Hierarchy**Patient ID Treatments Received**

P1	Healthcare, Cardiology, ECG
P2	Healthcare, Orthopedics
P3	Healthcare, Cardiology
P4	Healthcare, Cardiology, ECG
P5	Healthcare, Orthopedics

Question:

Develop pseudocode for **multilevel association rule mining** using the treatment hierarchy provided in the dataset. Explain how rules are generated at different hierarchy levels and how concept hierarchies improve pattern discovery.

ANSWER:

Q1. Apriori Algorithm — Online Shopping Transactions

Order ID	Products Purchased
O1	Mobile, Earphones
O2	Mobile, Charger
O3	Earphones, Charger
O4	Mobile, Earphones, Charger
O5	Mobile, Earphones

Assumption: minimum support threshold = 40% (count ≥ 2 out of 5 orders), used to illustrate the iterations below.

Pseudocode — Apriori Algorithm

ALGORITHM Apriori(Transactions D, min_support)

```
L1 = {frequent 1-itemsets found by scanning D and
      counting support of each item}
k = 2
WHILE L(k-1) is not empty DO
    Ck = CANDIDATE_GENERATION(L(k-1)) // self-join step
    Ck = PRUNE(Ck, L(k-1))           // Apriori-property pruning
    FOR EACH transaction t IN D DO
        Ct = subset of candidates in Ck contained in t
        FOR EACH candidate c IN Ct DO
            c.count = c.count + 1
        END FOR
    END FOR
    Lk = { c IN Ck | c.count / |D| >= min_support }
    RESULT = RESULT UNION Lk
    k = k + 1
```

```

END WHILE
RETURN RESULT

FUNCTION CANDIDATE_GENERATION(L(k-1))
    Ck=empty set
    FOREACH pair (p, q) IN L(k-1) x L(k-1) DO
        IF p and q share the first (k-2) items
            AND last item of p < last item of q THEN
                candidate = p JOIN q           // join step
                Ck= Ck UNION {candidate}
            ENDIF
        ENDFOR
    RETURN Ck

FUNCTION PRUNE(Ck, L(k-1))
    FOR EACH candidate c IN Ck DO
        FOR EACH (k-1)-subset s of c DO
            IF s NOT IN L(k-1) THEN
                REMOVE c FROM Ck           // prune step
                BREAK
            END IF
        END FOR
    END FOR
    RETURN Ck

```

Trace on the Online Shopping Dataset

Iteration k = 1 (candidate generation: scan D directly):

Item	Count	Support %	In L1?
Mobile	4	80%	Yes
Earphones	4	80%	Yes
Charger	3	60%	Yes

Iteration k = 2 — CANDIDATE_GENERATION self-joins L1 (all pairs, since $k-2 = 0$ items must match, which is vacuously true),

giving $C2 = \{\text{Mobile, Earphones}\}, \{\text{Mobile, Charger}\}, \{\text{Earphones, Charger}\}$.

PRUNE checks that both size-1 subsets of each candidate are in L1 — all pass, since L1 contains all three items.

Candidate (C2)	Count	Support %	In L2?
Mobile, Earphones	3 (O1,O4,O5)	60%	Yes
Mobile, Charger	2 (O2,O4)	40%	Yes
Earphones, Charger	2 (O3,O4)	40%	Yes

Iteration k = 3 — CANDIDATE_GENERATION joins L2 pairs sharing their first item, producing one candidate: {Mobile, Earphones, Charger}. PRUNE checks its three 2-subsets — {Mobile, Earphones}, {Mobile, Charger}, {Earphones, Charger} — all are in L2, so the candidate survives pruning and proceeds to support counting.

Candidate (C3)	Count	Support %	In L3?
Mobile, Earphones, Charger	1 (O4)	20%	No — fails min_support

Since L3 is empty, the algorithm terminates. Final frequent itemsets: $L1 = \{\text{Mobile, Earphones, Charger}\}$; $L2 = \{\text{Mobile, Earphones}\}, \{\text{Mobile, Charger}\}, \{\text{Earphones, Charger}\}$.

This trace illustrates both pruning mechanisms distinctly the Apriori-property (subset) prune, which would have blocked a candidate before counting

if any subset were infrequent, and the support-count elimination, which removes {Mobile, Earphones, Charger} only after counting because although all its subsets are frequent, its own support (20%) falls below the 40% threshold.

Q2. FP-Growth Algorithm — Library Book Borrowing Records (BL4)

Borrow ID	Books Borrowed
B1	Data Science, Python
B2	Python, Machine Learning
B3	Data Science, Machine Learning
B4	Data Science, Python, Machine Learning
B5	Python, Machine Learning

Assumption: minimum support = 40% (count ≥ 2 out of 5 records).

Pseudocode — FP-Growth Algorithm

```
ALGORITHM FP_Growth(Transactions D, min_support)
```

```
    //Step1: Build global header table
```

```
    F={item : count} for every item, scanning D once
```

```
    F={item IN F | count  $\geq$  min_support}    // frequent items only
```

```
    SORTF in descending order of count    // L order
```

```
    //Step2: Build the FP-Tree
```

```
    CREATE root node R (null)
```

```
    FOREACH transaction t IN D DO
```

```
        remove infrequent items from t
```

```
        SORT remaining items of t by order in F
```

```
        INSERT_TREE(t, R)
```

```
    ENDFOR
```

```
    RETURN FP_GROWTH_MINE(FP-Tree, null, min_support)
```

```
FUNCTION INSERT_TREE(itemList, node)
```

```
    p = first item in itemList
```

```
    IF node has a child N such that N.item = p THEN
```

```

    N.count = N.count + 1
ELSE
    CREATE new child N under node, N.item = p, N.count = 1
    LINK N into header table for p (node-link chain)
END IF
IF item List has more items THEN
    INSERT_TREE(rest of item List, N)    // recursive descent
END IF

FUNCTION FP_GROWTH_MINE(Tree, suffix, min_support)
    IF Tree contains a single path P THEN
        FOREACH combination C of nodes in P DO
            GENERATE pattern (C UNION suffix)
                with support = min count in C
        END FOR
    ELSE
        FOREACH item a in the header table of Tree
            (processed bottom-up) DO
                pattern = a UNION suffix
                GENERATE pattern with support = a.count
                build conditional pattern base of a
                (prefix paths of all nodes labelled a)
                build conditional FP-Tree from that base
                IF conditional FP-Tree is not empty THEN
                    FP_GROWTH_MINE(conditional Tree, pattern,
                                min_support)    // recursion
                END IF
            END FOR
        END FOR
    END IF
END FUNCTION

```

Building the FP-Tree on the Borrowing Dataset

Item counts: Python = 4 (B1,B2,B4,B5), Machine Learning = 4 (B2,B3,B4,B5), Data Science = 3 (B1,B3,B4). All three meet the 40% threshold. Frequency order (descending count, ties broken alphabetically): Python, Machine Learning, Data Science.

Borrow ID	Original Items	Reordered by Frequency
B1	Data Science, Python	Python, Data Science
B2	Python, Machine Learning	Python, Machine Learning
B3	Data Science, Machine Learning	Machine Learning, Data Science
B4	Data Science, Python, Machine Learning	Python, Machine Learning, Data Science
B5	Python, Machine Learning	Python, Machine Learning

Inserting each reordered transaction into the tree (INSERT_TREE) builds:

Root

```
|-- Python:4
|   |-- Data Science:1      (from B1)
|   `-- Machine Learning:3  (from B2, B4, B5)
|       `-- Data Science:1   (from B4)
`-- Machine Learning:1      (from B3)
    `-- Data Science:1
```

Header table (item -> total count, linked to all matching nodes): Python:4, Machine Learning:4 (3 under the Python branch + 1 on the separate branch), Data Science:3 (1+1+1 across three nodes).

Why No Candidate Itemsets Are Generated

Unlike Apriori, FP-Growth never explicitly enumerates and support-counts candidate itemsets. It compresses the database into the FP-Tree in two scans, then mines patterns directly by walking node-links and building small conditional

pattern bases / conditional FP-trees per item (e.g. the conditional base for Data Science is the set of prefix paths leading to it: {Python:1}, {Python, Machine Learning:1}, {Machine Learning:1}). Frequent patterns are grown by combining an item with its conditional tree's frequent items recursively (FP_GROWTH_MINE), so the search is guided by the tree structure rather than by generating and testing an exponential set of candidates as Apriori does.

Q3. Association Rule Generation — Frequently Purchased Services (BL3)

Service Combination	Support Count
{Internet}	4
{Streaming}	4
{Cloud Storage}	3
{Internet, Streaming}	3
{Streaming, Cloud Storage}	2

Assumption: total number of transactions = 4 (the count of the most frequent single item), and minimum confidence threshold = 70%.

Pseudocode — Rule Generation from Frequent Itemsets

```
ALGORITHM Generate_Rules(FrequentItemsets F, min_confidence)
```

```
    Rules = empty set
```

```
    FOREACH frequent itemset I IN F where |I| ≥ 2 DO
```

```
        FOR EACH non-empty proper subset A of I DO
```

```
            B = I - A                                // consequent
```

```
            confidence = SUPPORT_COUNT(I) / SUPPORT_COUNT(A)
```

```
            IF confidence ≥ min_confidence THEN
```

```
                RULE r = (A → B)
```

```
                r.support = SUPPORT_COUNT(I) / total_transactions
```

```
                r.confidence = confidence
```

```
                Rules = Rules UNION {r}                // keep rule
```

```
            END IF
```

```
        END FOR
```

```
    END FOR
```

```
    RETURN Rules
```

```
FUNCTION SUPPORT_COUNT(itemset X)
```

```
    RETURN precomputed support count of X from F
```

// (looked up directly; no re-scan of the transaction DB
// since counts were already produced by the mining step)

Applying It to the Given Service Combinations

Candidate Rule	Confidence Calculation	Confidence	Kept ($\geq 70\%$)?
Internet $\rightarrow$ Streaming	3/4	75%	Yes
Streaming $\rightarrow$ Internet	3/4	75%	Yes
Streaming $\rightarrow$ Cloud Storage	2/4	50%	No
Cloud Storage $\rightarrow$ Streaming	2/3	66.7%	No

Final Filtered Rules

- Internet $\rightarrow$ Streaming (support 75% of Internet orders also include Streaming; support = $3/4 = 75\%$ relative, overall support = $3/4$ total = 75%; confidence = 75%)
- Streaming $\rightarrow$ Internet (confidence = 75%)

Both remaining candidate rules — Streaming $\rightarrow$ Cloud Storage and Cloud Storage $\rightarrow$ Streaming — are discarded by the filtering step because their confidence (50% and 66.7% respectively) falls below the 70% threshold, even though {Streaming, Cloud Storage} was itself a frequent itemset. This shows that itemset frequency (support) and rule strength (confidence) are independent filters applied in sequence.

Q4. Constraint-Based Frequent Itemset Mining — Student Course Enrollment (BL4)

Student ID	Courses Enrolled
S1	Python, Database
S2	Python, AI
S3	Database, AI
S4	Python, Database, AI
S5	Python

Constraint: only itemsets containing "Python" are of interest. Assumption: minimum support = 40% (count ≥ 2 out of 5 students).

Pseudocode — Constrained Apriori (Constraint Pushed into Candidate Generation)

```
ALGORITHM Constrained_Apriori(D, min_support, constraint_item)
  L1 = {frequent 1-itemsets from D}
  // Do NOT filter L1 itself -- items other than the
  // constraint item are still needed to build valid
  // Python-containing itemsets in later joins.
  k = 2
  RESULT = empty set
  IF constraint_item NOT IN L1 THEN
    RETURN RESULT          // early exit: constraint unsatisfiable
  END IF
  WHILE L(k-1) is not empty DO
    Ck = CANDIDATE_GENERATION(L(k-1))
    Ck = PRUNE(Ck, L(k-1))    // Apriori-property prune
    Ck = { c IN Ck | constraint_item IN c }  // ** constraint push **
    COUNT support of every c IN Ck by scanning D
    Lk = { c IN Ck | c.count / |D| >= min_support }
```

```

    FOREACH itemset x IN Lk DO
        IF constraint_item IN x THEN
            RESULT = RESULT UNION {x}           // keep only qualifying
itemsets
        END IF
    ENDFOR
    k=k+ 1
    ENDWHILE
    RESULT = RESULT UNION {itemset {constraint_item} if frequent}
    RETURN RESULT

```

Trace on the Enrollment Dataset

Item	Count	Support %	Frequent?
Python	4	80%	Yes
Database	3	60%	Yes
AI	3	60%	Yes

At $k = 2$, the constraint is pushed in immediately after the join/prune step:

by the candidate {Database, AI} is generated
 CANDIDATE_GENERATION but is discarded before any support counting
 because it does not contain "Python". Only {Python, Database} and {Python, AI}
 survive to be counted.

Candidate (constrained C2)	Count	Support %	Frequent?
Python, Database	2 (S1,S4)	40%	Yes
Python, AI	2 (S2,S4)	40%	Yes

At $k = 3$, joining {Python,Database} and {Python,AI} produces {Python,
 Database, AI}, which already contains Python so no candidate is dropped by the
 constraint at this step; however its count is only 1 (S4), i.e. 20% support, so it fails
 the minimum-support test and L_3 is empty.

Effect of the Constraint on Candidate Generation and Pruning

The constraint “itemset must contain Python” is anti-monotone with respect to set containment when combined this way, so it can be pushed directly inside the level-wise loop rather than applied only at the end. This has two effects: (1) it prunes candidates immediately after the join step, before the expensive database scan for support counting, cutting the work of counting $\{\text{Database}, \text{AI}\}$ entirely; (2) it keeps items like Database and AI in play at the 1-itemset level (they are not discarded outright) because they are still needed as building blocks to form valid Python-pairs — only the final multi-item candidates that omit Python are removed. The final constrained result is: $\{\text{Python}\}$ (80%), $\{\text{Python}, \text{Database}\}$ (40%), $\{\text{Python}, \text{AI}\}$ (40%); $\{\text{Python}, \text{Database}, \text{AI}\}$ is excluded for failing minimum support, not the constraint.

Q5. Multilevel Association Rule Mining — Hospital Treatment Hierarchy

Patient ID	Treatments Received
P1	Healthcare, Cardiology, ECG
P2	Healthcare, Orthopedics
P3	Healthcare, Cardiology
P4	Healthcare, Cardiology, ECG
P5	Healthcare, Orthopedics

Treatment Concept Hierarchy

- Level 1 (general): Healthcare
- Level 2 (mid-level): Cardiology, Orthopedics
- Level 3 (specific): ECG (a diagnostic procedure under Cardiology)

Pseudocode — Multilevel Association Rule Mining

ALGORITHM Multilevel_Apriori(D, Hierarchy H, min_support[])

//min_support[] gives a (typically decreasing) support

//threshold per hierarchy level -- deeper/more specific

//levels naturally have lower counts.

FOR level = 1 TO H.max_depth DO

 D_level = ENCODE_ITEMS(D, H, level)

 // replace each item in every transaction with its

 // ancestor concept at the current hierarchy level

 L_level = Apriori(D_level, min_support[level])

 RESULT[level] = L_level

 Rules[level] = Generate_Rules(L_level, min_confidence)

END FOR

RETURN RESULT, Rules

FUNCTION ENCODE_ITEMS(D, H, level)

```

FOR EACH transaction t IN D DO
  FOR EACH item i IN t DO
    t = t with i REPLACED BY ancestor(i, H, level)
  END FOR
END FOR
RETURN D          // generalized transactions for this level

```

Applying It to the Treatment Records

Level 1 (Healthcare): every one of P1-P5 has Healthcare, giving 100% support - a trivially true, uninformative rule, so the algorithm typically does not report rules at the root level.

Level 2 (Cardiology, Orthopedics): generalizing ECG up to Cardiology where relevant:

Item (Level 2)	Count	Support %
Cardiology	3 (P1,P3,P4)	60%
Orthopedics	2 (P2,P5)	40%

Level 3 (ECG, specific to Cardiology patients): ECG appears in P1 and P4, i.e. among the 3 Cardiology patients.

Rule	Support (of all 5 patients)	Confidence
Cardiology -> ECG	2/5 = 40%	2/3 = 66.7%

How Concept Hierarchies Improve Pattern Discovery

Mining level by level (top-down) lets the algorithm apply a different, appropriately-scaled minimum support at each level -- a high threshold at the general Healthcare level would hide nothing interesting since everyone matches it, while a lower threshold is needed at the specific ECG level, since specific procedures naturally occur less often than their broad parent category. By encoding items to their ancestor concept at each level (ENCODE_ITEMS) before

re-running Apriori, the same transaction data yields rules ranging from general (which are usually filtered out as trivial, e.g. Healthcare-related rules) to specific and actionable (Cardiology -> ECG), without needing separate, unrelated mining passes. This reveals a pattern -- most Cardiology patients also receive an ECG -- that would not be visible either at the fully general level (masked by universal Healthcare) or without any hierarchy at all (where ECG alone has too low a raw support to appear reliably as significant on its own).